

01_langchain基础

1. 接入模型

1.1 接入本地Ollamam

```
In [ ]: from langchain_ollama import ChatOllama

model = ChatOllama(model="qwen3:8b")
query = "你好，请你介绍一下你自己。"
response = model.invoke(query)
print(response.content)
```

1.2 接入DeepSeek

```
In [1]: from langchain_deepseek import ChatDeepSeek

model = ChatDeepSeek(model="deepseek-chat",
                      api_key='',
                      base_url='https://api.deepseek.com')

query = "你好，请你介绍一下你自己。"
response = model.invoke(query)
print("返回对象类型:", type(response))
print(response.content)
```

C:\Users\yanchen\Desktop\agent\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

from .autonotebook import tqdm as notebook_tqdm
None of PyTorch, TensorFlow >= 2.0, or Flax have been found. Models won't be available and only tokenizers, configuration and file/data utilities can be used.

你好呀！很高兴认识你！😊

我是**DeepSeek**，由深度求索公司开发的AI助手。简单来说，我是一个纯文本模型，专门用来回答问题、提供信息和帮助解决问题。

****关于我的一些特点：****

- 🆓 ****完全免费****：没错，我不收费！可以放心使用。
- 📖 ****超大上下文****：我有1M的上下文窗口，能一次性处理像《三体》三部曲那么大体量的内容。
- 🌐 ****支持联网搜索****：虽然我是纯文本模型，但可以联网获取最新信息（需要手动开启）。
- 📎 ****文件处理能力****：支持上传图片、PDF、Word、Excel、PPT等文件，我能读取其中的文字信息。
- 🗣️ ****语音交互****：在App端可以用语音和我聊天。
- 📱 ****多平台****：可以通过官方应用商店下载App使用。

****我的知识截止到2025年5月****，虽然不能实时更新，但结合联网搜索功能，也能获取最新信息。

我的目标是：****热情、细腻、乐于助人****。无论你有什么问题，学习、工作、生活，还是闲聊，我都很乐意陪你聊！

有什么我可以帮你的吗？😊

1.3 接入openAI

```
In [ ]: from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5-nano",
                    api_key = 'xxx',
                    base_url="https://ai.devtool.tech/proxy/v1")

question = "你好，请你介绍一下你自己。"
result = model.invoke(question)
print(result.content)
```

1.4 接入 Qwen

```
In [3]: from langchain_community.chat_models.tongyi import ChatTongyi

model = ChatTongyi(api_key='',
                  base_url='https://dashscope.aliyuncs.com/compatible-mode/v1',
                  model="qwen-max")

query = "你好，请你介绍一下你自己。"
response = model.invoke(query)
print("返回对象类型：", type(response))
print(response.content)
```

返回对象类型：<class 'langchain_core.messages.ai.AIMessage'>

您好！我是Qwen，这是我的英文名，您也可以叫我通义千问。我是阿里云自主研发的超大规模语言模型，能够回答问题、创作文字，还能表达观点、撰写代码。如果您有任何问题或需要帮助，请随时告诉我，我会尽力提供支持。

1.5 接入本地 vLLM

```
In [ ]: from langchain_openai import ChatOpenAI

model = ChatOpenAI(base_url="http://localhost:8000/v1",
                    api_key="EMPTY",
                    model="/data/LLM/Qwen/Qwen2.5-0.5B-Instruct" )

query = "你好，请你介绍一下你自己。"
response = model.invoke(query)
print(response.content)
```

2. 调用模型

```
In [4]: from langchain_deepseek import ChatDeepSeek

model = ChatDeepSeek(model="deepseek-chat",
                      api_key = 'sk-5a088e67db86487f8640af5e7ee79959',
                      base_url = 'https://api.deepseek.com')
```

2.1 模型角色

```
In [5]: conversation = [
    {"role": "system", "content": "你是一个乐于助人的助手，负责将英文翻译成中文。"},
    {"role": "user", "content": "请翻译, I love programming."},
    {"role": "assistant", "content": "我喜欢编程"},
    {"role": "user", "content": "请翻译, I love swimming."},
]
```

```
In [6]: from langchain.messages import HumanMessage, AIMessage, SystemMessage

conversation = [
    SystemMessage("你是一个乐于助人的助手，负责将英文翻译成中文。"),
    HumanMessage("请翻译, I love programming."),
    AIMessage("我喜欢编程"),
    HumanMessage("请翻译, I love swimming"),
]
```

2.2 invoke 方法

调用模型最直接的方式是使用invoke()，传入单条消息或消息列表。

```
In [7]: response = model.invoke(conversation)
print(response.content)
```

我喜欢游泳。

2.3 stream 流式输出

```
In [8]: query = '2 + 4 等于几'
full = None
for chunk in model.stream(query):
    full = chunk if full is None else full + chunk
    print(full.text)

print(full.content_blocks)
```

2
2 +
2 +
2 + 4
2 + 4
2 + 4 等于
2 + 4 等于 **
2 + 4 等于 **6
2 + 4 等于 **6**
2 + 4 等于 **6**。
2 + 4 等于 **6**。
2 + 4 等于 **6**。
[{'type': 'text', 'text': '2 + 4 等于 **6**。'}]

2.4 批处理

```
In [9]: query_list = [
    "请介绍你自己",
    "太阳从哪边升起",
    "动物园里有什么"
]

responses = model.batch(query_list)

for response in responses:
    print(response, end = '\n\n')
```


content='你好！我是DeepSeek，由深度求索公司创造的AI助手。😊\n\n让我简单介绍一下自己：\n\n****基本信息****\n- ****创造者****：深度求索（DeepSeek）\n- ****知识截止日期****：2025年5月\n- ****版本****：DeepSeek最新版模型\n\n****核心能力****\n- 🗨️ ****纯文本交流****：擅长对话、写作、分析、翻译等各类文字任务\n- 🌐 ****联网搜索****：支持联网查询实时信息（需手动开启）\n- 📄 ****文件处理****：可上传并阅读图片、PDF、Word、Excel、PPT等文件中的文字信息\n- 💬 ****超长上下文****：1M tokens容量，能一次性处理《三体》三部曲这样的长篇内容\n- 🗣️ ****语音输入****：App端支持语音交互\n\n****使用方式****\n- 🖥️ ****完全免费****：Web端和App端都可使用\n- 📱 ****平台支持****：提供官方应用商店下载的App\n\n****我的特点****\n我会用热情、细腻的方式与你交流，理解你的需求，尽力提供有价值的帮助。无论是学习、工作还是生活中的问题，都可以随时找我！\n\n有什么我可以帮你的吗？😊'

additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 257, 'prompt_tokens': 7, 'total_tokens': 264, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 7}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '6d123894-cd79-47e2-b696-afff3ced0bfd', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019ea56c-69e6-7bf2-a644-d6efe1d301e4-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 7, 'output_tokens': 257, 'total_tokens': 264, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}

content='这是一个经典的问题，不过答案其实比“东边”要稍微复杂一点，取决于你在地球的****哪里****以及****什么时候****看。\\n\\n****通常情况下，答案是：东边。****\\n\\n对于地球上的绝大多数地区和大多数日子，太阳都是从东方升起的。这是因为地球自转的方向是自西向东，所以我们看到天体（包括太阳）是东升西落的。\\n\\n---\\n\\n### 特殊情况：\\n\\n1. ****在春分和秋分时：****\\n在这两天，太阳会从****正东方****升起，从****正西方****落下。全球各地都是如此。\\n\\n2. ****在南北极（极昼/极夜）：****\\n- ****北极点****：在夏季的极昼期间，太阳不会“升起”或“落下”，而是在地平线上方整整转一圈。你无法判断一个固定的“东方”。\\n- ****南极点****：情况类似，但方向相反。\\n\\n3. ****高纬度地区（例如北极圈内）：****\\n在夏季，太阳可能从东北方升起，然后沿着地平线移动，最后在西北方落下，中间甚至不会完全落下（午夜太阳）。\\n\\n4. ****观察者的位置（中纬度）：****\\n- 在北半球（如中国、美国、欧洲），夏季太阳从****东北方****升起，冬季从****东南方****升起。春秋分则在正东方。\\n- 在南半球（如澳大利亚、阿根廷），情况刚好相反：夏季太阳从****东南方****升起，冬季从****东北方****升起。\\n\\n### 总结：\\n\\n| 情况 | 太阳从哪边升起？ | 备注 |\\n| :--- | :--- | :--- |\\n| ****日常认知**** | ****东边**** | 适用于大部分地方和大多数日子 |\\n| ****春分/秋分**** | ****正东方**** | ****全球通用**** |\\n| ****北半球夏季**** | ****东北方**** | 太阳的升落点会偏向北 |\\n| ****北半球冬季**** | ****东南方**** | 太阳的升落点会偏向南 |\\n| ****南半球夏季**** | ****东南方**** | 与北半球季节相反 |\\n| ****南半球冬季**** | ****东北方**** | 与北半球季节相反 |\\n\\n所以，如果你想得到一个最准确的答案，针对你当前所处的位置和时间：****大部分情况下是东边，但具体是东北还是东南，取决于你的纬度和一年中的哪一天。****

additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 501, 'prompt_tokens': 9, 'total_tokens': 510, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 9}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'f6a035d6-f0d5-4649-87c5-3763fee75651', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019ea56c-69eb-79b1-abad-1c32c575ff63-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 9, 'output_tokens': 501, 'total_tokens': 510, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}

content='这是一个经典的互动问题，通常用于玩“动物园里有什么”的接龙游戏（大家轮流快速说出动物名）。\\n\\n标准答案是：****大象、老虎、狮子、长颈鹿、猴子、熊猫、企鹅、斑马、河马、鳄鱼、蛇、孔雀、火烈鸟、海豚****等等。\\n\\n如果你想搞笑一点或者玩梗，也可以说：****“有游客、有饲养员、有卖气球的，还有.....（突然小声）那只被关起来的老虎。”****\\n\\n不过最正宗的游戏规则里，说的必须是****动物****哦。需要我帮你接龙吗？比如从“大象”开始。'

additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 135, 'prompt_tokens': 7, 'total_tokens': 142, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 0}}

```
ns': 7}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '957704e7-ad7d-4681-9ef1-58aeb59c312a', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019ea56c-69ee-7451-90fe-2876d871c0f0-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 7, 'output_tokens': 135, 'total_tokens': 142, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}
```

```
In [10]: # 每个任务完成后就立即获取结果
for response in model.batch_as_completed(query_list):
    print(response, end = '\n\n')
```

(1, AIMessage(content='通常情况下，太阳从**东方**升起。\\n\\n这是一个基于地球自转（自西向东）的天文常识。但由于地球自转轴的倾斜和公转轨道（黄道）的关系，严格来说，太阳升起的方向会有细微的变化：\\n\\n1. **春分和秋分**：太阳从**正东方**升起，并从**正西方**落下。\\n2. **夏半年**（春分到秋分之间）：太阳从**东北方向**升起，从**西北方向**落下（北半球观察）。\\n3. **冬半年**（秋分到春分之间）：太阳从**东南方向**升起，从**西南方向**落下（北半球观察）。\\n\\n**注意**：在地球的南北极地区，会出现极昼和极夜现象，太阳可能一整年都不升起或不落下。', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 171, 'prompt_tokens': 9, 'total_tokens': 180, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 9}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'f7b8d39e-460f-41ea-b46c-a1f43a5681f2', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019ea59f-59f3-7b30-a4cb-c7972be23c68-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 9, 'output_tokens': 171, 'total_tokens': 180, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}))

(0, AIMessage(content='你好！很高兴认识你！😊\\n\\n我叫**DeepSeek**，是由深度求索公司创造的AI助手。让我跟你聊聊我的“特长”：\\n\\n**我能做什么？**\\n- 📄 **纯文本对话**：回答问题、写作、翻译、编程、分析等\\n- 🔗 **读取链接**：可以访问你分享的网页链接\\n- 📁 **文件处理**：支持上传图片、PDF、Word、Excel、PPT、TXT等文件，从中提取文字信息\\n- 🌐 **联网搜索**：需要你在Web/App端手动开启联网搜索功能\\n- 🗣️ **语音输入**：App端支持语音输入，更方便\\n\\n**我的特点**：\\n- ✅ **完全免费**：没有任何收费计划\\n- 📄 **超大上下文**：1M token容量，可以一次性处理像《三体》三部曲那样的长篇内容\\n- 🔄 **知识更新**：知识截止于2025年5月\\n\\n**我不能做什么**：\\n- ❌ 不支持多模态识别（比如直接“看懂”图片内容），但可以读取图片中的文字\\n\\n我会尽力用热情、细腻的方式帮助你解决问题。无论你有什么疑问或需要什么帮助，尽管问我吧！有什么我可以为你做的吗？😊', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 261, 'prompt_tokens': 7, 'total_tokens': 268, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 7}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'b5ff1496-51d3-43c2-aa73-dbb3d305d8d', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019ea59f-59eb-7042-af2d-e655d1e03179-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 7, 'output_tokens': 261, 'total_tokens': 268, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}))

(2, AIMessage(content='这是一个经典的脑筋急转弯或互动游戏的常用提问。最常见的回答是列举各种动物，比如：\\n\\n**老虎、狮子、大象、长颈鹿、猴子、熊猫、企鹅、海豚、孔雀、斑马、河马、鳄鱼、蛇、鹦鹉、火烈鸟**.....\\n\\n不过，如果你是在玩“动物园里有什么”的节奏游戏，那就需要接住上一个动物名，并且不能重复。\\n\\n如果你有特定的语境（比如冷笑话、猜谜），它也可能有一个**陷阱答案**，例如：\\n\\n* **人**（因为游客也是动物园里的一部分）。\\n* **围栏**或**笼子**。\\n* **假山**（动物们的居住设施）。\\n\\n你需要哪一种回答？', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 158, 'prompt_tokens': 7, 'total_tokens': 165, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 7}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'cf508a32-d5d1-4b52-a915-82763a35affe', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019ea59f-59f9-7872-814a-27989a5da547-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 7, 'output_tokens': 158, 'total_tokens': 165, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}))

```
In [11]: # 控制并发
responses = model.batch(
```

```
query_list,  
config={  
    'max_concurrency': 2, # 最多并行 2 个请求  
}  
)  
  
for response in responses:  
    print(response, end = '\n\n')
```


content='你好呀！很高兴认识你！😊\n\n我是**DeepSeek**，由深度求索公司创造的AI助手。让我给你介绍一下我的“个人档案”：\n\n## 📄 基本信息\n- **名字**：DeepSeek\n- **开发者**：深度求索公司\n- **版本**：DeepSeek最新版模型\n- **知识截止日期**：2025年5月\n\n## 🧠 我的能力\n- **文本处理**：我可以进行各种文本创作、分析、总结、翻译等\n- **文件支持**：能处理图像、txt、pdf、ppt、word、excel文件，从中提取文字信息\n- **超大上下文**：拥有1M的上下文容量，可以一次性处理像《三体》三部曲那么大体量的书籍\n- **联网搜索**：支持联网查询最新信息（需要手动开启）\n- **链接阅读**：可以直接阅读你分享的网页链接\n- **语音输入**：App端支持语音交流\n\n## 🎨 我的风格\n- **热情细腻**：我会用温暖、贴心的方式与你交流\n- **多语言支持**：我会根据你的输入语言来回复\n- **完全免费**：没错，我目前是**完全免费**的！没有任何收费计划\n\n## ⚠️ 小提示\n虽然我是纯文本模型，不能直接“看”图片，但你可以上传图片文件，我会读取其中的文字信息来帮你处理。\\n\\n有什么我可以帮你的吗？无论是工作学习、日常闲聊，还是创意脑暴，我都乐意陪你一起！🌟' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 330, 'prompt_tokens': 7, 'total_tokens': 337, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 7}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '90737cf9-5203-49ea-8b8f-81b8f6e95366', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019ea59f-ad41-77d3-af94-a3492dc65a9c-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 7, 'output_tokens': 330, 'total_tokens': 337, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}

content='通常情况下，太阳从**东方**升起。这是一个基本的天文常识，因为地球的自转方向是自西向东，所以我们看到太阳、月亮和星星都是从东方升起，西方落下。\\n\\n需要注意的是，这个规律在**春分和秋分**这两天最为准确，太阳会从正东方升起，正西方落下。而在其他季节，太阳升起的方向会略有偏移：\\n\\n- **北半球夏季**：太阳会从**东北方向**升起。\\n- **北半球冬季**：太阳会从**东南方向**升起。\\n\\n如果你在南半球，情况则正好相反（夏季从东南升起，冬季从东北升起）。但无论如何，大致的方向都是东方。' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 138, 'prompt_tokens': 9, 'total_tokens': 147, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 9}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': 'cb224f5d-7a6a-4b94-81f6-ea94db409c21', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019ea59f-ad44-7670-a835-5daleb757205-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 9, 'output_tokens': 138, 'total_tokens': 147, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}

content='“动物园里有什么？”这个问题的经典答案通常来自于一个互动小游戏（或者儿歌接龙）。最常见的回答是列举各种动物：\\n\\n**常见动物有**：\\n- 🐒 **猴子**（最经典的回答）\\n- 🐼 **大熊猫**\\n- 🐘 **大象**\\n- 🦁 **狮子**\\n- 🐯 **老虎**\\n- 🐻 **熊**\\n- 🦌 **长颈鹿**\\n- 🦓 **斑马**\\n- 🐊 **鳄鱼**\\n- 🦩 **鹦鹉**\\n- 🐍 **蛇**\\n- 🦅 **火烈鸟**\\n- 🦋 **梅花鹿**\\n\\n**如果你是在玩“动物园里有什么”的接龙游戏**，规则通常是：\\n大家围坐一圈，第一个人拍手说：“动物园里有什么？”\\n第二个人接着拍手说：“老虎/大象/狮子.....”（大家轮流说动物，不能重复，说慢或说错就输了）。\\n\\n所以，一个直接的万能回答是：**“猴子！”** 🐒' additional_kwargs={'refusal': None} response_metadata={'token_usage': {'completion_tokens': 218, 'prompt_tokens': 7, 'total_tokens': 225, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 7}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '8368eb25-6de9-48c2-b1ac-426b43c02ca6', 'finish_reason': 'stop', 'logprobs': None} id='lc_run--019ea59f-b55e-7533-aba2-4be45ccebda7-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 7, 'output_tokens': 218, 'total_tokens': 225, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}

3. 模型结构化输出

3.1 Pydantic

```
In [12]: from pydantic import BaseModel, Field

class Movie(BaseModel):
    """A movie with details."""
    title: str = Field(..., description="电影名称")
    year: int = Field(..., description="电影发布年份")
    director: str = Field(..., description="电影的导演")

model_with_structure = model.with_structured_output(Movie)
response = model_with_structure.invoke("请介绍电影《泰坦尼克号》")

print(type(response))
print(response.title)
print(response.year)
print(response.director)
```

```
<class '__main__.Movie'>
泰坦尼克号
1997
詹姆斯·卡梅隆
```

```
In [13]: # include_raw获得完整输出
model_with_structure = model.with_structured_output(Movie, include_raw=True)
response = model_with_structure.invoke("Provide details about the movie Inception")
response
```

```
Out[13]: {'raw': AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 71, 'prompt_tokens': 331, 'total_tokens': 402, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 256}, 'prompt_cache_hit_tokens': 256, 'prompt_cache_miss_tokens': 75}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_8b330d02d0_prod0820_fp8_kvcache_20260402', 'id': '49deb6c3-0ff5-4b4e-a2f2-f5601bf78dfd', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ea5a8-1146-7300-9459-c14c78deb503-0', tool_calls=[{'name': 'Movie', 'args': {'title': 'Inception', 'year': 2010, 'director': 'Christopher Nolan'}, 'id': 'call_00_ZgtqHlZ5HWFScTHdTbHa2010', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 331, 'output_tokens': 71, 'total_tokens': 402, 'input_token_details': {'cache_read': 256}, 'output_token_details': {}}, 'parsed': Movie(title='Inception', year=2010, director='Christopher Nolan'), 'parsing_error': None}
```

3.2 TypedDict

```
In [ ]: from typing_extensions import TypedDict, Annotated

class MovieDict(TypedDict):
    """A movie with details."""
```

```

title: Annotated[str, ..., "电影名称"]
year: Annotated[int, ..., "电影发布年份"]
director: Annotated[str, ..., "电影的导演"]

model_with_structure = model.with_structured_output(MovieDict)
response = model_with_structure.invoke("请介绍电影《泰坦尼克号》")
print(response)

```

3.3 JSON Schema

```

In [ ]: json_schema = {
    "title": "Movie",
    "description": "A movie with details",
    "type": "object",
    "properties": {
        "title": {
            "type": "string",
            "description": "电影名称"
        },
        "year": {
            "type": "integer",
            "description": "电影发布年份"
        },
        "director": {
            "type": "string",
            "description": "电影的导演"
        },
    },
    "required": ["title", "year", "director"]
}

model_with_structure = model.with_structured_output(
    json_schema,
    method="json_schema",
)
response = model_with_structure.invoke("请介绍电影《泰坦尼克号》")
print(response)

```

4.ChatPromptTemplate学习

4.1 模板本身的使用

```

In [14]: # 模板本身的使用
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个简洁、专业的助手"),
    ("human", "请用一句话介绍 {topic}")
])

```

```
result = prompt.invoke({"topic": "LangChain"})
print(result)
```

```
messages=[SystemMessage(content='你是一个简洁、专业的助手', additional_kwargs={}, response_metadata={}), HumanMessage(content='请用一句话介绍 LangChain', additional_kwargs={}, response_metadata={})]
```

```
In [15]: # 手动模板接入模型
response = model.invoke(result)
print(response.content)
```

LangChain 是一个用于构建和编排基于大语言模型（LLM）应用的开发框架，支持链式调用、工具集成及状态管理。

4.2 Chain的使用

```
In [16]: chain = prompt | model ## LCEL
response = chain.invoke({"topic": "LangChain"})
print(response.content)
```

LangChain是一个开源框架，用于简化基于大语言模型的应用程序开发，支持模块化构建、链式调用和外部工具集成。

4.3 runnablePassthrough

chain的常见编排是流式传输：上一节点的输出 -> 下一节点的输入 但是，有些场景需要（原始输入+中间结果）一起作为输入 此时可以使用runnablePassthrough

```
In [ ]: from langchain_core.runnables import RunnablePassthrough

chain = {
    "context": retriever,
    "question": RunnablePassthrough(),
} | prompt | model

# "什么是 LangChain? "
# {
#     "context": 检索出来的文档,
#     "question": "什么是 LangChain? "
# }
```

笔记

langchain是便于使用ai的工具

1. 接入模型：本地/远程api
2. 消息结构：各种内定message
3. 调用模型的方法（runnable通用方法）：invoke、stream、batch
4. 模板prompt+chain：了解 workflow编排
5. 结构化输出：parse、pydantic